

MKT 智能体平台 · 平台主手册系列 · 之二

Vibe Coding 速查手册

必备技能 50 条 · 工作流衔接 · 多智能体架构 · 服务器部署 · AI 问题对策 · 生成工具选型

为 Joe 定制 · 2026年7月

与《Vibe Coding 实战手册》配套：实战手册讲原理和学习路径，本册是桌边速查——表格化、结论先行、拿来即用。

工具盘点与价格信息截至 2026-07，时效性强，建议每季度刷新。

目录

1	Vibe Coding 必备技能 50 条	按七个主题分组：下任务/上下文/验证/调试/安全/成本/心态
2	工作流衔接与编排速查	两层分离、幂等、checkpoint、handoff、人审节点、排空部署
3	多智能体架构速查	什么时候单 agent 就够、编排模式、上下文工程、失败模式
4	服务器部署实操速查	systemd/nginx/TLS/零停机/防火墙/日志轮转 checklist
5	AI 使用常见问题与对策	幻觉/漂移/注入/成本/限流/模型选型/输出不稳定
6	生图 · 生视频 · 生PPT 工具矩阵	2026-07 主流工具盘点：场景→推荐→理由；平台现有接入评估

第1章 Vibe Coding 必备技能 50 条

从 Anthropic 官方最佳实践、Karpathy/Willison 方法论、2025-2026 社区共识清单中精编，每条=一个可执行动作+为什么。按七个主题分组，全部条目有出处、其中多条已在你的平台上被事故验证。

一、下任务 (1~9)

1. **四段式工作流：探索→规划→实现→提交**。多文件改动/不熟的代码/方案不确定必走规划；diff 能一句话描述就直接干——规划本身也有开销。
2. **任务拆小到单个可验证增量 (Karpathy"拴紧狗链")**。AI 生成速度已超过人的验证速度，瓶颈在验证端；任务越大验证面越大。
3. **提示词三要素：哪个文件 + 什么场景/约束 + 什么算做完**（含要跑的验证命令）。"修登录bug"是坏提示；"用户报X，查 auth 目录，先写失败复现测试再修，转绿为准"是好提示。
4. **指向参考实现而非描述风格**。"参考 XxxWidget.js 的模式实现，不引入新库"——一个好的参考文件胜过一百句风格描述。
5. **大功能先让 AI 反向面试你，产出 SPEC 再开新会话执行**。"详细面试我，挖我没想到的难点，写成 SPEC.md"；spec 必含涉及文件、明确的 out-of-scope、端到端验证步骤。
6. **先要 2~3 个方案选项，拍板后再要代码**。避免第一个方案锚定后返工——与"懂架构不写码"的角色天然契合。
7. **修 bug 给三件套：症状原文 + 疑似位置 + "修好"的定义**。先复现再修是防"修了个寂寞"的根本手段（你的"别臆断根因"规矩同源）。
8. **声明禁改区并在临近时重复**。鉴权、支付、存储格式列为 off-limits 写进规则文件；干相邻活时提示词里再重复一次。
9. **改动前要 blast-radius**。让 AI 先报"将改哪些区段+可能波及的功能"再放行——对 3 万行大文件尤其（你的"先析背景禁单点隧道"规矩即此条）。

二、上下文管理 (10~17)

10. **上下文窗口是最稀缺资源，填满即降智**。18 个前沿模型实测全部随输入变长退化，中段信息准确率可掉 30%+；AI 变笨先看上下文占用，别先怪模型。
11. **规则文件只写 AI 猜不到的，逐行做删除测试**。"删掉这行 AI 会犯错吗？不会就删"；AI 屡教不改先怀疑文件太长，而不是加更多规则（你的"指令稿瘦身"规矩同源）。
12. **两次纠正无效就清空重开**。失败历史污染上下文，越纠越偏；把教训写进新 prompt 第一句，胜过在废墟上继续修。
13. **探索性阅读交给子代理**。子代理读几十个文件只汇报摘要，主会话上下文留给实现。
14. **超大文件给一张常驻代码地图**。区段索引（第X段=路由、第Y段=存储）写进规则文件；禁止对大文件做整文件重写式编辑。
15. **换任务就清上下文，同任务深挖不清**。防止上一个功能的架构假设渗透到下一个。
16. **领域知识做成按需加载的 skill，不塞常驻规则**。判据："每个会话都用到吗？"否→skill（你的 ainspace-* skills 体系即此条）。
17. **喂原始数据**。贴报错全文/截图/linter 输出，禁止口头转述堆栈——原始数据形成 AI 可自纠的调试环。

三、验证与验收 (18~27)

18. **给 AI 一个它能自己跑的检查——看守型与放手型会话的分水岭**。没有可跑的检查，"看起来完成"就是 AI 唯一的停止信号，你本人就成了验证环节。
19. **要证据不要宣称**。验收话术固定为"贴出测试输出/你跑的命令和返回"，不接受纯文字"已修复"。
20. **改一次验一次：改动→typecheck→测试→commit 四步一拍**。多个未验证改动叠加后，失败无法定位。
21. **无测试的老代码先补特征测试锁行为**。没有行为锁的区域禁止 AI 重构；这类测试可以丑、用完即弃也值。

- 22. **UI 改动用截图对比验收。**贴目标图→实现→AI 截图对比列差异；深浅色两套各截一次（你的 UI 规矩同源）。
- 23. **对抗式评审：新鲜上下文的第二个 AI 只看 diff 挑错。**限定“只报影响正确性的 gap”——被要求挑错的评审者总会报点什么，照单全收导向过度工程。
- 24. **diff 固定扫四项：**被删文件是否有人引用、公共接口签名是否变了、新增依赖是否必要且真实存在、有无硬编码密钥。
- 25. **测试是不可外包的责任。**可以让 AI 写测试，但“验证代码确实能工作”不可外包——你没验证过的功能等于不存在。
- 26. **专项猎杀吞异常。**grep 空 catch 块和“只打日志不上抛”；点验全过但功能没发生的事故几乎都源于此。
- 27. **验收标准由你独立定义。**绝不让写代码的 AI 自拟自判——实现和验收出自同一会话时，bug 会被“验证”成正确行为。

四、调试与止损 (28~33)

- 28. **修根因不压症状。**修复类 prompt 必带“不许压制错误或跳过测试”；review 看到 skip/放宽校验一律打回。
- 29. **识别 doom loop：同一 bug 第 3 次修复尝试→停手。**revert 到最后一个好提交，用吸收了教训的新 prompt 一次做对——补丁摞补丁是标志性事故。
- 30. **高频 commit 当存档点，信息写给未来的 AI 看。**一个可验证增量=一个 commit；日后让 AI“读 git history 总结这段代码为什么这样写”。
- 31. **给 AI 运行时的“眼睛”。**前端 bug 让它看浏览器 console/network，后端 bug 先喂 journalctl 切片——静态猜远不如实证。
- 32. **卡死换模型。**单模型失败 2 轮→把问题+已试方案打包丢给另一家；不同模型盲区不同，这是免费的第二意见。
- 33. **前端报失败≠任务失败。**长任务先查后端产物和用量账本再判定，盲目重跑=重复烧钱（平台生视频事故实证）。

五、安全红线 (34~41)

- 34. **密钥永不进代码；进过 git 历史就视为已泄露，立即轮换。**定期全库 grep sk-/api_key/secret/password。
- 35. **安全必须写进提示词。**AI 默认优化“能跑”不优化“安全”；涉及输入/存储/网络的任务固定附加安全条款。
- 36. **逐路由做“未登录调用会怎样”测试。**脚本枚举全部路由无凭据请求一遍，非 401/403 即漏洞——vibe-coded 应用最常见高危项。
- 37. **输入处理路径加倍审。**实测 AI 代码引入 XSS 概率是人写的 2.74 倍；用户输入 > 外部数据解析 > 数据库写入，三查：空值/形状假设/错误是否被吞。
- 38. **致命三角逐 agent 盘点。**私密数据+不可信内容+对外通信三者齐备=现成外泄通道，至少砍一条；防御靠架构不靠“教 AI 别上当”。
- 39. **新依赖核实三点：**真实存在、下载量/维护正常、不是知名包的变体拼写（防幻觉包与 slopsquatting）。
- 40. **schema/存储格式变更先要迁移+回滚方案再要代码。**数据不可逆是最大单点风险；生产数据操作前先备份。
- 41. **SAST+密钥扫描进流水线。**人审跟不上 AI 产码量，机器扫机器写的代码。

六、成本与效率 (42~45)

- 42. **模型分级路由：旗舰做规划/评审，中档写码，经济档干杂活。**从中档开始，卡住才升档；成本差 10~20 倍。
- 43. **外部服务优先 CLI 而非堆 MCP。**MCP 工具定义常驻消耗上下文；AI 能通过 --help 自学陌生 CLI。
- 44. **批量机械任务先试 3 个再全量。**无头模式逐文件 fan-out，按小样本翻车情况改 prompt 再放量。
- 45. **“必须每次发生”的规则做成 hook/门，不靠嘱咐。**判据：违反一次就出事吗？是一脚本门；否→留在规则文件（你的门体系即此条的平台化）。

七、心态与角色 (46~50)

- 46. **把 AI 当“过度自信的天才实习生”**（Karpathy 原话）：知识百科全书级，但会一本正经胡说、勇气过剩；**置信语气与正确率无关**——这个心智模型决定你所有验证强度的设置。

47. **思考可以外包，理解不能。**只合并你能解释的 diff；讲不清就让 AI 讲到你懂，或重写成你懂的方案——解释不了的代码出事时你修不了。
48. **并行上限 = 你能认真审查的数量**（多数人 2~3 个）。互相可能改同一文件的任务禁止并行；以吞吐量论成败是 90 年代代码行数指标的重演。
49. **止损规则前置。**单任务 N 轮未收敛即触发换路：换模型/回滚/改拆法/降级目标，不跟单一会话死磕。
50. **你的角色是审代码+设计循环。**重复出现的"prompt→检查→修正"序列固化成脚本化循环+质量门，人只处理被门拦下来的异常——这是 2026 年从"英雄式提示词"到"loop engineering"的共识方向，也正是你平台渲染门/部署门已经在走的路。

✅ **只记三条的话**

#2（任务拆小拴紧狗链）、#18（给 AI 能自己跑的检查）、#46（过度自信的实习生）。其余 47 条都是这三条在不同场景下的展开。

第2章 workflow 衔接与编排速查

workflow 引擎几十年沉淀的原则在 AI agent 时代依然成立。核心一句话：编排逻辑与副作用步骤分层，副作用一律幂等、可重试、有心跳，状态落盘、终态不可变。

2.1 十二条设计原则

原则	要点	落地做法
1. 两层分离（Temporal 核心思想）	确定性编排层（同输入必产生同一步骤序列）与副作用执行层（API 调用/写盘）分开；恢复=重放编排代码，已成功步骤直接取历史结果	编排代码里禁止直接 fetch/随机数/取当前时间；副作用收进步骤函数，结果落盘
2. 幂等是重试的前提	没有幂等的重试=复制副作用（重复扣费/重复建任务）；队列必有重复投递，幂等做在消费侧	幂等键=任务ID+阶段号；创建前先查存在；删除时 not-found 视为成功
3. 一张任务表+不可变终态	最小状态机： pending→working→completed/failed/cancelled， 进终态后不可变 ；预期内失败建模为"带错误码的完成"	用户重试=创建新任务行，不是复活旧行；前端报失败先查表和产物（账本才是真相）
4. checkpoint 卡阶段边界	每微步都存拖慢执行，只存内存进程死即丢；共识粒度=每次工具调用/阶段边界	恢复前先验证 checkpoint 引用的产物文件真实存在； 任务干净跑完必须清 checkpoint ，否则下次全新运行误跳步
5. handoff 传简报不传全史	子任务收"目标+输出格式+边界"，大产物写外部存储只传引用，只返回蒸馏摘要（Anthropic 多智能体系统实践）	阶段间交接="上阶段写产物+manifest，下阶段读 manifest"；别把大 JSON 塞进队列消息
6. 编排优于协作（单团队规模）	中央编排器流程可预测、易排查；事件协作扩展灵活但全局状态难追踪	你的任务队列+状态表就是编排器；失败时间"哪些已发生的副作用需要抵消"（Saga 补偿思想）
7. 人审只拦不可逆高爆炸半径	逢步必审→人变瓶颈→审批疲劳瞎批；审批门放动作 前 （放在非确定性步骤后，恢复重跑会产生与已批内容不同的结果）	人审清单：生产部署/删数据/花钱超阈值/对外发送；审批消息带"将执行什么+diff 预览"
8. 轮询为权威，推送为加速	webhook 会静默丢失、重复、乱序；轮询浪费但可靠	内网低频→纯轮询；用户在等→轮询打底+SSE 加速；跨系统→webhook+定期扫"卡在 working 超时"的任务对账
9. 长步骤要心跳+带进度	心跳区分"还在干"和"已死透该重试"，避免慢被误杀成死	worker 每 30s 更新任务行 heartbeat_at+当前进度；扫描器只在心跳超时才判死重派
10. 部署要排空在飞任务	更新代码打断跑一半的任务→用户侧诡异失败（Anthropic 用 rainbow 部署：新旧并存，旧版本跑完在飞任务才退役）	单机降级方案：停派新任务→等在飞终态→部署→恢复（你的夜间窗+排空正是此模式）
11. 批处理三原则（Airflow 沉淀）	原子（每任务独立可重跑）、幂等（跑 N 次=跑 1 次）、增量（按逻辑时间切片）	定时产物按逻辑日期命名（report-2026-07-07.md），重跑覆盖同名天然幂等，补跑=换日期
12. 从简单模式起步	五个可组合模式：链式/路由/并行/编排者-工人/评审-优化循环；拆多 agent 需有隔离性收益论证	步骤可预定义→链式；子任务依输入而变→编排者-工人；有明确评价标准→评审-优化（=你的渲染门 loop）

2.2 高频踩坑表

错误	后果	纠正
checkpoint 只存内存	进程死=任务凭空消失	外部持久化（任务表/文件）
HTTP 请求生命周期和执行循环绑死	连接断="失败"，后端其实还在跑	提交立即返回 task_id，执行全异步
重试没有幂等垫底	重复扣费/重复写入	幂等键+创建先查+消费侧去重
只信 webhook 不对账	静默丢事件，任务永远"进行中"	轮询为权威+定期扫超时任务
终态可回写	重试期间状态互相覆盖，账本失真	终态进入即冻结
硬栅栏式并行（等全员到齐）	单个卡死拖死整批	子任务独立超时+容错聚合，缺席者标记降级
过早拆多 agent	prompt/trace/审批面翻倍，能力没涨	单 agent 起步

📖 平台对照

你的平台已经踩实了几条：生视频"断连后端续跑+账本对账"= 原则3；夜间窗排空 = 原则10；指令稿驱动 codex = 原则5的文件化实现（稿=任务简报，回执=轻引用）。差距项：任务表缺统一 heartbeat 字段（原则9）、checkpoint 清理纪律（原则4）、"卡在 working 超时"的对账扫描器（原则8）。2026-07-07 两条研究 workflow 因硬栅栏并行被单 agent 卡死拖挂 2 小时，正是踩坑表最后第二条的现场实证。

第3章 多智能体架构速查

2025-2026 业界在多智能体上的最大共识是一句反直觉的话：**多数场景，单 agent + 好工具 + 好上下文，优于多 agent**（Anthropic《Building Effective Agents》与 Cognition《Don't Build Multi-Agents》双双背书）。拆分是有成本的决策，不是先进性的象征。

3.1 什么时候才值得拆多 agent

值得拆（有隔离性收益）	不值得拆
任务天然可并行且互不依赖（多主题调研、多文件独立迁移）	"感觉更高级"、想复刻别人的架构图
上下文隔离有价值：脏探索留在子 agent，主 agent 只收结论	子任务之间高度耦合、要频繁互相通信（错误会级联放大）
能力/权限隔离：处理不可信内容的 agent 降权只读（切致命三角）	单 agent 加一个工具就能解决
读写分离：多个只读研究者可以并行， 多个写者不行 （会互踩）	预算敏感场景——Anthropic 实测多 agent 系统 token 消耗约为单聊天的 15 倍

3.2 五个可组合的编排模式（从简到繁，够用即止）

模式	适用	平台对应物
链式 （Prompt Chaining）	步骤可预定义的固定流水线	PPT 的"搜索→分析→规划→生成→自评"
路由 （Routing）	按输入类型分发给不同处理者	助手任务模式把生图/生视频/代码分发给不同执行器
并行 （Parallelization）	互不依赖的子任务同时跑	多主题调研 fan-out（注意容错聚合，见 2 章踩坑表）
编排者-工人 （Orchestrator-Workers）	子任务数量和内容依输入而变	玉华拆稿→codex 队列执行，即此模式的人机版
评审-优化循环 （Evaluator-Optimizer）	产物有明确评价标准	skill-loop 引擎（GLM 生成+vision 评审）、RenderSight 渲染门

选型口诀：**步骤固定用链式，类型分发用路由，量大互独用并行，动态拆分用编排者，有标准就上评审循环**——五个都不够用再考虑自由协作式多 agent（几乎轮不到）。

3.3 上下文工程：多 agent 的成败处

- **任务简报式交接**：子 agent 收"目标+输出格式+工具指引+明确边界"，不是完整对话史；缺边界的子 agent 会重复劳动或跑偏（Anthropic 实测教训）。
- **产物走引用**：大产出写外部存储，agent 间只传轻量引用——内容反复复制会劣化+撑爆下游窗口。
- **子 agent 只回蒸馏摘要**：内部烧几万 token 探索可以，返回控制在千 token 级。
- **Cognition 两原则**：共享完整上下文（含决策依据），行动携带隐式决策——两个写者各自基于不完整上下文行动，冲突不可避免（你的"codex 与监督端改同一文件互踩"事故即此）。推论：**写者唯一化，或先抢锁**。

3.4 通信协议：MCP 与 A2A 一句话分工

MCP (Anthropic, 2024 起已成事实标准) : agent ↔ 工具/数据源的连接协议——“给 agent 插 USB”。**A2A** (Google 2025 发起, 已入 Linux 基金会) : agent ↔ agent 的互操作协议, 跨厂商 agent 互相发现和委托任务——生态早期, 单平台内部编排用不上。你的场景结论: 内部编排继续用自研任务表+文件通道 (够用且可控), 对外接工具时优先找 MCP server; A2A 观望即可。

3.5 可观测与评估

- **trace 是多 agent 的生命线**: 每个 agent 的输入简报、工具调用、返回、耗 token 都要落盘 (你的 journal/回执机制同构) ——没 trace 的多 agent 系统故障时等于黑箱套黑箱。
- **评估看终局也看过程**: 终局对不对 (golden set) + 过程是否异常 (某 agent 重试暴涨、耗时离群、返回为空)。
- **失败模式三巨头**: 错误级联 (上游 agent 的错误输出被下游当事实)、成本爆炸 (无预算门的递归拆分)、责任不清 (出错后不知道该修哪个 agent 的 prompt)。对策依次: 关键交接处设校验门、给编排层硬预算上限、每个 agent 单一职责+独立可测。

📖 平台对照

你的平台架构在业界坐标系里是“**中央编排者-工人 + 评审循环**”的组合, 且编排层是确定性代码而非模型自由发挥——这恰好是 Anthropic 推荐的稳妥形态 (确定性编排优于模型自治)。两个已被本地事故验证的守则: ①并行研究型 agent 必须独立超时+容错聚合 (2026-07-07 两条 workflow 被单 agent 卡死 2 小时); ②多写者必须抢锁 (codex/监督端互踩事故)。一个提升项: 给编排层加 per-任务 token 预算与告警, 防成本爆炸。

✅ 学习路径 (3 小时读完全部必需品)

①Anthropic 《Building Effective Agents》(1h, 五模式地图+“从简单开始”论证); ②Anthropic 《How we built our multi-agent research system》(1h, 生产级多 agent 最密集的一手经验: 简报交接/token 经济学/rainbow 部署); ③Cognition 《Don't Build Multi-Agents》(0.5h, 反方必读, 上下文两原则); ④MCP 概念页扫一眼 (0.5h)。**不必学**: LangGraph/AutoGen/CrewAI 等框架 API——你的自研编排已工作, 框架迁移是负资产; A2A 细节——生态未稳。

第4章 服务器部署实操速查（单机/小规模）

单机 Node 生产部署的业界共识已收敛。本章按子系统给“复制即用”的结论，全部结论核实于 2025-2026 一手来源。

4.1 进程管理：已在 systemd 上就别装 PM2

systemd 自带崩溃重启、开机自启、资源限制、journald 日志，零额外依赖；PM2 的真实增量只有免代码多核 cluster 和监控面板——单进程应用叠 PM2 等于两层进程管理器互相打架，属反模式。**Restart 语义正解**（man 页核实）：无论 always 还是 on-failure，**手动 systemctl stop 都不会被拉起**（“stop 后又活了”要查 timer/其他管理器/部署脚本）；两者区别是 always 连正常退出也重启。长跑服务用 on-failure。

✅ 复制即用的加固模板（目标 systemd-analyze security <4 分）

```
[Service]
User=appuser           # 专用非 root 用户，80/443 由 nginx 占
Restart=on-failure
RestartSec=5
StartLimitBurst=5      # 防 crash loop 刷屏
NoNewPrivileges=yes    # 禁提权
ProtectSystem=strict   # 文件系统只读
ReadWritePaths=/srv/app/data # 白名单放行写路径（文件型存储关键）
PrivateTmp=yes
ProtectHome=yes
EnvironmentFile=/etc/app/app.env # chmod 600
```

硬性特例：Node/Bun 等 JIT 运行时永远不加 MemoryDenyWriteExecute=yes——V8 的 JIT 会被内核拒绝，进程启动即崩死。价值：即使被 RCE，攻击者也写不了白名单外的磁盘。

4.2 nginx 反代四大坑

坑	正解
真实 IP	客户端可伪造 X-Forwarded-For 头；首层代理必须取信 TCP 建连 IP： <code>proxy_set_header X-Real-IP \$remote_addr</code> ，后端一律读 X-Real-IP。验收： <code>curl -H "X-Forwarded-For: 1.2.3.4"</code> 打一发，确认日志记录的不是 1.2.3.4
超时语义	<code>proxy_read_timeout</code> 是“两次读之间的最大间隔”不是总时长——后端持续有输出就不断， 静默计算超 60s 就被掐断报 504 （长任务“前端报中断、后端成功”的典型根因之一）。长计算接口给 300~600s 且后端发心跳。502=连不上后端，504=连上了但超时，别混
大文件	<code>client_max_body_size</code> 默认仅 1m，超了直接 413（“小文件能传大文件必挂”）；调大同时配 <code>proxy_request_buffering off</code> 流式直达，防 nginx 先整包缓冲
SSE/流式五件套	<code>proxy_buffering off; proxy_cache off; proxy_http_version 1.1; proxy_set_header Connection ""; gzip off;</code> + 长 <code>read_timeout</code> 。漏一个就“卡住不出字”。偷懒法：后端对流式响应统一加 <code>X-Accel-Buffering: no</code> 头，不用改 nginx。验收： <code>curl -N</code> 看是否逐段到达

4.3 TLS：证书寿命正在从 200 天砍到 47 天

CA/B Forum 已定死时间表：公网证书最长寿命 2026-03 起 200 天 → 2027-03 100 天 → 2029-03 47 天；Let's Encrypt 2027-02 默认降到 64 天。**含义：手动续期在这个周期下必然出事故，自动化是生存条件。**最常见事故不是“没续”而是“续了没 reload”——nginx 不 reload 就继续用旧证书到期。

✔ 一次配好

先 `systemctl list-timers | grep certbot` 查有无自带 timer (apt/snap 装的通常有), 补 deploy-hook: `certbot renew --deploy-hook "nginx -t && systemctl reload nginx"`; 装完立刻 `certbot renew --dry-run` 验证, 之后每季度 dry-run 一次防链路腐化。自签证书会导致浏览器长连接异常断开 (平台已实证), 公网一律换真证书。

4.4 零停机: 优雅退出三步曲

SIGTERM → ①健康检查改返 503 (上游停发新流量) → ② `server.close()` 停收新连接 → ③等在飞排空, 超时强杀。
keep-alive 陷阱: 老版本 Node 的空闲长连接让进程退不掉, 最后被 SIGKILL 硬杀丢请求; Node 18.2+ 用原生 `closeIdleConnections()` / `closeAllConnections()`, 不必再引第三方包。systemd 配 `TimeoutStopSec=45` (比应用超时略长)。单机零停机走 nginx upstream 双实例: 起新实例→冒烟→切 upstream→reload (nginx reload 本身平滑) →停旧实例。**生成类长任务平台必须先排空在飞任务再停旧实例。**

4.5 原子切换与"半新半旧"

`ln -sfn` 不是原子操作 (unlink+symlink 两步, 中间有空窗报 ENOENT); 正解 `ln -s releases/NEW tmp && mv -T tmp current`。禁止 rsync 直接覆盖运行中目录——那会造出"磁盘新代码+内存旧进程"的半新半旧状态, 反复出诡异 bug。**判断生产状态只认完整部署成功回执, 不认"文件已就位"** (平台记忆"本地就绪≠已上线"同源)。保留最近 5 个 release, 回滚=原子切回+重启, 秒级; 回滚演练至少做一次, 别等事故时第一次用。

4.6 Docker 决策口诀: 一个应用不 Docker, 三个以上异构服务才 compose

Docker 的收益是环境一致性和多服务编排, 代价是多一层引擎维护 + 著名安全坑: **-p 发布的端口完全绕过 ufw** (Docker 直写 iptables FORWARD 链, "以为防火墙挡了数据库实际全网裸奔"是高频真实事故)。你的形态 (单机+单 Node 应用 + 已有 systemd/nginx/原子部署门) 加 Docker 是纯增熵。若将来用: 仅本机的端口一律 `-p 127.0.0.1:5432:5432` 绑回环。

4.7 防火墙、日志、磁盘、备份: 五件做完封 90% 暗坑

子系统	收口做法
防火墙	<code>ufw default deny incoming; ufw limit 22; ufw allow 80,443/tcp</code> ; 应用统一 listen 127.0.0.1 由 nginx 反代套认证; 每月 <code>ss -tlnp grep 0.0.0.0</code> 巡检公网监听面 (41218 教训的制度化)。改 SSH 规则前先开第二个会话防锁死自己
journald	默认能默默吃掉数 GB: 配 <code>SystemMaxUse=1G + MaxRetentionSec=14day</code> ; 验证 <code>journalctl --disk-usage</code>
logrotate	应用文件日志 daily/rotate 14/compress; 注意 create 模式要求应用重开 fd (否则新文件永远空), copytruncate 有丢日志窗口——按天写新文件的应用天然免疫
磁盘监控	磁盘打满是单机最常见"整站静默故障"根因 (写不进文件→任务失败→日志也写不进)。一条 cron 够: 使用率 >80% 告警接现有通道; 生成类产物目录 (图/视频/PPT) 增长快于日志, 单独设保留天数定期清
备份	OS 整盘快照≠应用容灾 (同区域同账号, 防不了误删后知后觉/勒索/账号被黑, 且做不到"只找回昨天误删的文件")。restic 异地: 每日备数据目录+env+nginx/systemd 配置+证书 (产物可只备近期); 每日 <code>forget --prune</code> ; 每月抽样恢复比对, 每季度全量演练——从未恢复过的备份等于没有备份 ; 备份密码另存一处

📖 平台对照：按优先级的差距清单

1. **restic 应用级异地备份**——评审 F45 确认目前应用层零备份，这是最大裸奔面。
2. Let's Encrypt 替换自签证书（整改路线图已列，长连接误报的诱因）。
3. SIGTERM 优雅退出 + closeIdleConnections（配合排空部署）。
4. systemd 单元加固五指令（当前未加固，RCE 后果放大器）。
5. journald 上限 + 磁盘 80% 告警 + 产物目录保留策略。

✅ 学习路径（合计约 2 天，大半可交给编码代理执行）

①30 分钟精读一份 systemd 生产模板，会 `systemctl status` + `journalctl -u -f` 即可；②半天只学 nginx proxy 十来个指令，用自己平台的一个 SSE 接口和一个上传接口各调通一次；③1 小时跑通证书自动链路；④半天补优雅退出+原子切换（可交代理）；⑤半天做防火墙/日志/磁盘/备份基线，之后按季度复查。**可以不学**：K8s、服务网格、Prometheus 全家桶——单机场景投入产出为负。

第5章 AI 使用常见问题与对策速查

十个高频问题区，没有一个能靠“写更好的提示词”根治，全部要靠工程手段兜底。核心心法：**信任产物前先看验证门——AI 的输出与自我报告一律过机器可判定的校验，人只看被拦下来的东西。**

5.1 十大问题速查表

问题	事实与机理	对策
幻觉包名/API	约 19.7% 的 AI 推荐包不存在（商业模型约 5.2%），43% 幻觉名稳定复现可被抢注（slopsquatting）。口诀： AI 说有≠有，注册表说有才算有	新依赖脚本核验存在性+创建日期+下载量（新包<3月且低下载=默认拒）；lockfile 锁死；编 API 靠“先跑通再交付”
上下文衰减	Chroma 实测 18 个模型：长上下文中段信息准确率可掉 30%+，远未填满窗口就开始衰减；可用上下文按标称的 25~30% 规划	一个会话只干一件事，任务切换就重开；进度/决策落盘到文件，新会话从文件恢复；代理重复问已答过的问题=立即重开别硬续
Prompt injection	OWASP LLM Top 10 2025 第一名。防护提示词“拦截 95%”在安全上等于不及格。唯一可靠防御是架构级：切断 致命三角 （私有数据+不可信内容+对外通道）	给每个 agent 画三角检查表，三项全勾必须拆；优先做出网域名白名单；处理不可信内容的代理降权只读
行为性故障	谎报完成（Replit 事故：无视禁令 11 次、伪造测试数据、删生产库）、应付测试（硬编码期望输出/改测试）、改坏无关处。 代理的自我报告不可信	验收契约三件套：机器可判定验收标准；测试对代理只读；diff 超出验收范围即告警回退
成本失控	头号失败是只看总账单不归因。缓存读约为输入价 0.1×（省 90%），但前缀逐字节匹配——system prompt 嵌时间戳/随机 ID 会让缓存 永不命中还多付写入溢价	每调用记一行账本{user, feature, model, tokens, cost}；小时花费>7日基线 2 倍告警；开缓存后用 cache_read_tokens 验证真命中
429/限流	指数退避+jitter+封顶是标准配方；SDK 通常已内置重试， 自己再包一层=故障时流量翻倍	只重试 429/5xx/超时；400/422 不重试修 prompt；401/403 不重试查 key；Retry-After 当下限
模型选型	轻量档比旗舰便宜 10~20 倍且分类/抽取够用；cascade（便宜先行，低置信升级）稳定优于单一路由	抽取/分类/改写→轻量；CRUD 代码/文档→中档；跨文件重构/根因分析/指令稿→旗舰。降档用 golden set 对比，掉点<2% 即切
输出不稳定	裸 prompt 要 JSON 失败率 5~15%；schema 强制模式压到<0.5%。但 schema 合规≠语义正确 ：合规率>97% 的模型字段值错误率仍达 17~31%	一律 schema 强制模式+扁平结构（嵌套≤3层）；后面必须再加业务校验层（枚举白名单/数值区间/ID 真实存在）
缺评估	自顶向下造指标=高分+生产翻车； 自底向上（从真实失败倒推）才预测真实故障	从历史 bug 挑 20~50 条建 golden 回归集接部署门；每次事故加一条；LLM-as-judge 只用于没法机器判定的（文案/版式），且 judge 换模型防自恋
供应商单点	地域封锁/限流突变/审核收紧都能让无备份应用瘫痪（你的 OpenAI 出口 403 事故实证）； 换道的前提是调用层本来就抽象了 （codex→GLM 切换成功正因如此）	调用收敛到统一 callLLM 封装：fallback 链+熔断+per 任务候选；生图生视频等不可替代能力标记“无备份依赖”并给用户降级提示

5.2 高频踩坑补充

- 把 API key/密码写进提示词或代理会话——会话历史会持久化、可被接口读出、可经注入外流；密钥只走 env/vault 通道（你的"密码由 codex 直写文件不经玉兔"规矩同源）。
- 多写手（多代理/人+代理）并行改同一文件不加锁——互相覆盖、推半成品、冲掉热修；写手唯一化或先抢锁是硬规则。
- 长任务前端报错就判失败、不查后端产物先重跑——重复烧钱（平台生视频误报已实证，先查账本）。
- 生产事故不沉淀成评估用例——等于让用户永远当测试员。

✅ 学习路径（半数可交给编码代理落地）

① Simon Willison 《The lethal trifecta》30 分钟——AI 安全唯一必需心智模型；② Anthropic 《Effective context engineering》1 小时——compaction/外部笔记/子代理三杠杆；③ 交给代理落地工程三件套（统一 callLLM+账本告警+标准退避），1~2 天，覆盖成本/限流/容灾 90% 收益；④ 半天建 golden 回归集+验收契约模板。**够用线**：学习型路由器、语义缓存、评估平台属于"日均万次调用再考虑"的深度。

第6章 生图 · 生视频 · 生PPT 工具矩阵（截至 2026-07）

先说结论：平台现有四路接入（OpenAI Image2 + 方舟 Seedream + Seedance + 可灵）就是 2026 年各家评测“默认推荐清单”的交集，没有任何一路需要替换。缺口只有三个按需项。本章价格时效性强，引用时注意抓取日期。

6.1 生图格局

工具	定位与数据	选型结论
GPT Image 2（已接）	2026-04 发布即断层第一：Arena Elo 1512，领先第二名 242 分（史上最大分差）；强在复杂指令遵循和多图编辑。按 token 计费：1024 ² 图约 low \$0.006 / medium \$0.053 / high \$0.211—— 档位差 35 倍	主力不换。省钱在质量档：按用途给默认档，别全走 high
Nano Banana 2（Google）	榜二（Elo 1360），照片级真实感口碑强	海外出口轨，暂无需接
Seedream（已接）	中文语义/人像强、低价补位；5.0 Lite ¥0.22/图 已上线	同渠道平滑升级：方舟控制台核对 <code>doubao-seedream-5.0-lite</code> 模型 ID
Ideogram 4.0	图内文字渲染第一（海报标语/logo/多行排版），2026-06 开源权重+商用许可，官方 API 与自托管双轨；支持 JSON 版面控制（bounding-box）	现有组合唯一明显短板 ：用户常做带文字海报时补接，先官方 API
Recraft V3	独有原生 SVG/矢量输出和品牌风格锁定（约 \$0.04/图）	仅当有 SVG/品牌素材需求
FLUX.2	LoRA 微调生态事实标准（dev 32B 开源）；SD 主要剩存量 ComfyUI 价值	仅当有微调/自托管需求
Midjourney	审美仍顶级，但 仍无公开 API （企业版限量申请制）；市面“MJ API”全是违反 ToS 的 Discord wrapper，账号随时被封	只做人肉灵感来源，不进代码 ；要 MJ 风格用 FLUX.2/Seedream 加风格化提示词逼近

6.2 生视频格局

工具	定位与价格	选型结论
Seedance（已接）	三强之一。 价格纠偏 ：网传 \$0.022/秒是 1.x lite 旧价；2.0 方舟官方约 ¥1/秒（15s 720p 约 ¥14），能力升级但价格上探一个量级；1.x lite 仍是批量低价主力	默认/批量走 1.x 低价档；升级 2.0 前 必须实测 API 开放状态 （公布定价时曾标注暂不支持 API）
可灵 Kling 3.0（已接）	官方 V3 API 已开放：原生 4K、15 秒、音频/口型同步；官方预付资源包制，第三方 \$0.075/秒起	4K/口型/带音频需求走它；控制台确认 V3 模型权限
Veo 3.1（Google）	Fast \$0.15/s、Standard 含音频 \$0.40/s；音画同步和企业合规口碑最好	企业级音画成片需求时补接（走 Gemini API， 先验生产服务器出口链路 ）
Sora	已死 ：App 2026-04-26 关停，API 2026-09-24 停服（日烧 \$100 万收入仅 \$210 万）	勿接勿依赖；grep 代码确认无 sora 端点。教训：闭源生成模型有整体下线风险， 视频永远保持双供应商 ——你现有 Seedance+可灵正是正确容灾结构
Vidu Q2 / Runway / Pika	Vidu 低价备胎（约 \$0.0375/s）；Runway/Pika 是 credit 订阅制创作工具	订阅制计费模型不匹配按量平台，不做 API 后端

6.3 生 PPT 格局

工具	定位	选型结论
自研路线（现状）	平台需要品牌模板+RenderSight 渲染门控制，市面无等价物	保留，路线正确
Gamma	Generate API 2026-01 GA—— 唯一主流"生成即API" 的通用 deck 工具（Pro \$15-20/月起，credit 制），综合能力评测第一（12 页约 20 秒）	花 \$20 开一个月跑 API 对标，把它的排版结构当自研质量基准
Kimi PPT / WPS AI / iSlide / 百度文库	国产强在中文汇报； 可编辑性/导出 pptx 保真度参差是最大坑 ；仅 iSlide 支持私有化/API	轻量替代标注 Kimi 免费档；选型必实测导出而非看预览

6.4 两条横向规则

⚠ 渠道与合规双轨

海外模型（OpenAI/Google）不对中国大陆服务，需出口链路+美元支付，出口 IP 可能被地域封锁（你的 OpenAI 403 事故实证）；国产渠道（方舟/可灵/百炼）人民币、国内直连、走备案合规。**新接海外模型前先在生产服务器验证出口连通，再写代码。**聚合平台（fal/Replicate/EvoLink）只用于评测对标，不做生产主路径（有加价、快照滞后、多一层依赖）。

✅ 比价必先统一单位

计费单位五花八门（/图、/百万像素、/token、/秒、/credit、/席位），不换算就对比会误判几倍到几十倍（本次调研审校就抓到一例把 Seedance 1.x 旧价当 2.0 现价、低估 6~10 倍）。**统一换算成两个锚点：「元/张 1K 图」和「元/5 秒 720p」**，写进 usage 账本口径并标注价格日期。2026 年中锚点速记：生图 \$0.006~0.21/张；生视频 5 秒 \$0.19（Vidu）~ \$2.0（Veo Standard），跨度约 10 倍。

🗉 给决策者的一句话

不换主力，不接 Sora，不碰 Midjourney wrapper；方舟控制台核对 seedream-5.0-lite 与 Seedance 2.0 开放状态；Ideogram（图内文字）/Veo（音画成片）/Gamma（PPT 对标）三个按需项等真实用户需求出现再接，每项评估半天足够。保鲜方式：收藏 Arena 榜、OpenAI pricing、方舟价格页、可灵计费页四个"活页面"，**每季度刷一次。**